

Entregable 6

1. Resumen de Pruebas de Validación

Tabla de resultados obtenida tras la ejecución de los 6 casos de prueba definidos para el TokenManager. Las pruebas se realizaron mediante ejecución controlada y simulación de escenarios de red y concurrencia.

Caso	Descripción	Resultado Esperado	Resultado Obtenido	¿Coincide?
1	Token bien formado, no expirado	is_expiring_soon() = False	False	Sí
2	Token expirado hace 10 min	is_expiring_soon() = True	True	Sí
3	Token malformado (2 partes)	decode_payload() = None	None	Sí
4	Payload inválido (No JSON)	decode_payload() = None	None	Sí
5	Token sin campo 'exp'	is_expiring_soon() = True	True	Sí
6	Refresh simultáneo (Singleton)	Solo 1 petición real al servidor	1 petición (Double-Check)	Si

**Nota sobre el Caso 6: El código funcionó lógicamente bajo condiciones ideales, pero se identificó un bug crítico en el manejo de excepciones que afectaba la resiliencia.*

2. Análisis de Bugs y Mejoras Aplicadas

Bug Identificado: "Fallo en Cascada por Error de Red"

Descripción: Durante la prueba del Caso 6, se observó que si el primer hilo en obtener el

candado (Lock) fallaba debido a un error de red (Timeout), el sistema ejecutaba automáticamente un logout().

Causa Raíz: El bloque try/except en la función refresh_access_token() no discriminaba entre tipos de errores. Al capturar cualquier Exception, el código procedía a borrar el refresh_token de la memoria. Esto impedía que los hilos subsiguientes que esperaban en el Lock pudieran intentar el refresh una vez restablecida la conexión, forzando un cierre de sesión innecesario para el usuario.

Corrección Aplicada:

- Se modificó el manejo de excepciones para distinguir entre errores de protocolo (HTTP 401 Unauthorized) y errores de infraestructura (ConnectionError/Timeout).
- **Lógica de Fix:** Solo se llama a self.logout() si el servidor responde explícitamente que las credenciales son inválidas. Si ocurre un error de red, el refresh_token se mantiene en memoria para permitir reintentos automáticos en la siguiente petición.

3. Conclusiones

La implementación actual del TokenManager es robusta frente a tokens malformados y eficiente en entornos concurrentes. La corrección del manejo de excepciones asegura que el sistema sea resiliente ante la inestabilidad de la red, manteniendo la sesión activa siempre que sea posible.